

Busca Competitiva



Gabriel P. Oliveira

Inteligência Artificial
2026/1

Problem-solving agents

Um tipo de agente que decide o que fazer procurando **sequências de ações que o leva a um objetivo** (estado desejado) para um dado problema

Problem-solving agents

Um tipo de agente que decide o que fazer procurando **sequências de ações que o leva a um objetivo** (estado desejado) para um dado problema

*O que fazer quando existe um **outro agente** que está atuando contra nós?*

1.

Teoria dos Jogos

Qual a relação com métodos de busca?

Teoria dos Jogos

- ▷ Ramo da **matemática aplicada** que visualiza qualquer ambiente multiagente como um jogo
- ▷ O impacto de cada agente sobre os outros deve ser significativo
 - Agentes podem ser **cooperativos** ou **competitivos**

Teoria dos Jogos

- ▷ Ramo da **matemática aplicada** que visualiza qualquer ambiente multiagente como um jogo
- ▷ O impacto de cada agente sobre os outros deve ser significativo
 - Agentes podem ser **cooperativos** ou **competitivos**

Jogos de soma zero → agentes agem alternadamente e os **objetivos são sempre opostos**

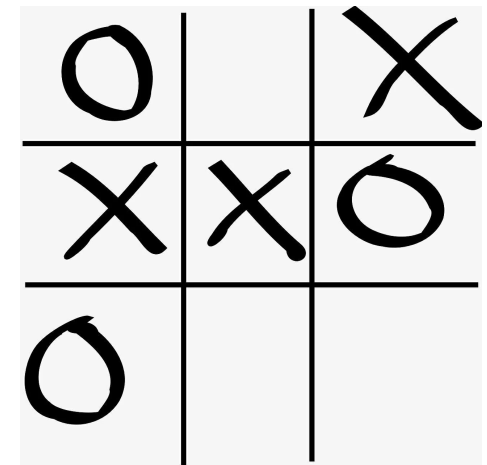
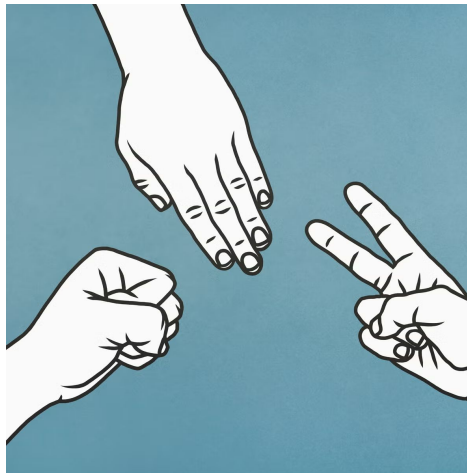
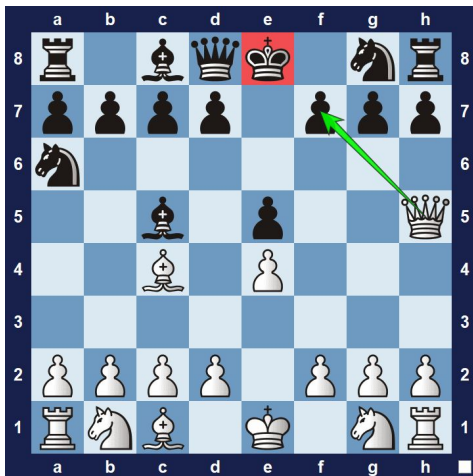
- ▷ O que é bom pra um, é ruim para o outro

Exemplos?

Teoria dos Jogos

Jogos de soma zero → agentes agem alternadamente e os objetivos são sempre opostos

- ▷ O que é bom pra um, é ruim para o outro



Teoria dos Jogos em IA

- ▷ Jogos são interessantes porque são **muito difíceis de resolver**
 - Xadrez tem *branching factor* médio de 35
 - Árvore de busca tem aproximadamente 10^{154} nós
- ▷ Jogos exigem a habilidade de **tomar alguma decisão**, até mesmo quando o cálculo da decisão ótima é inviável

Como modelar jogos como problema de busca?

Formulação do Problema

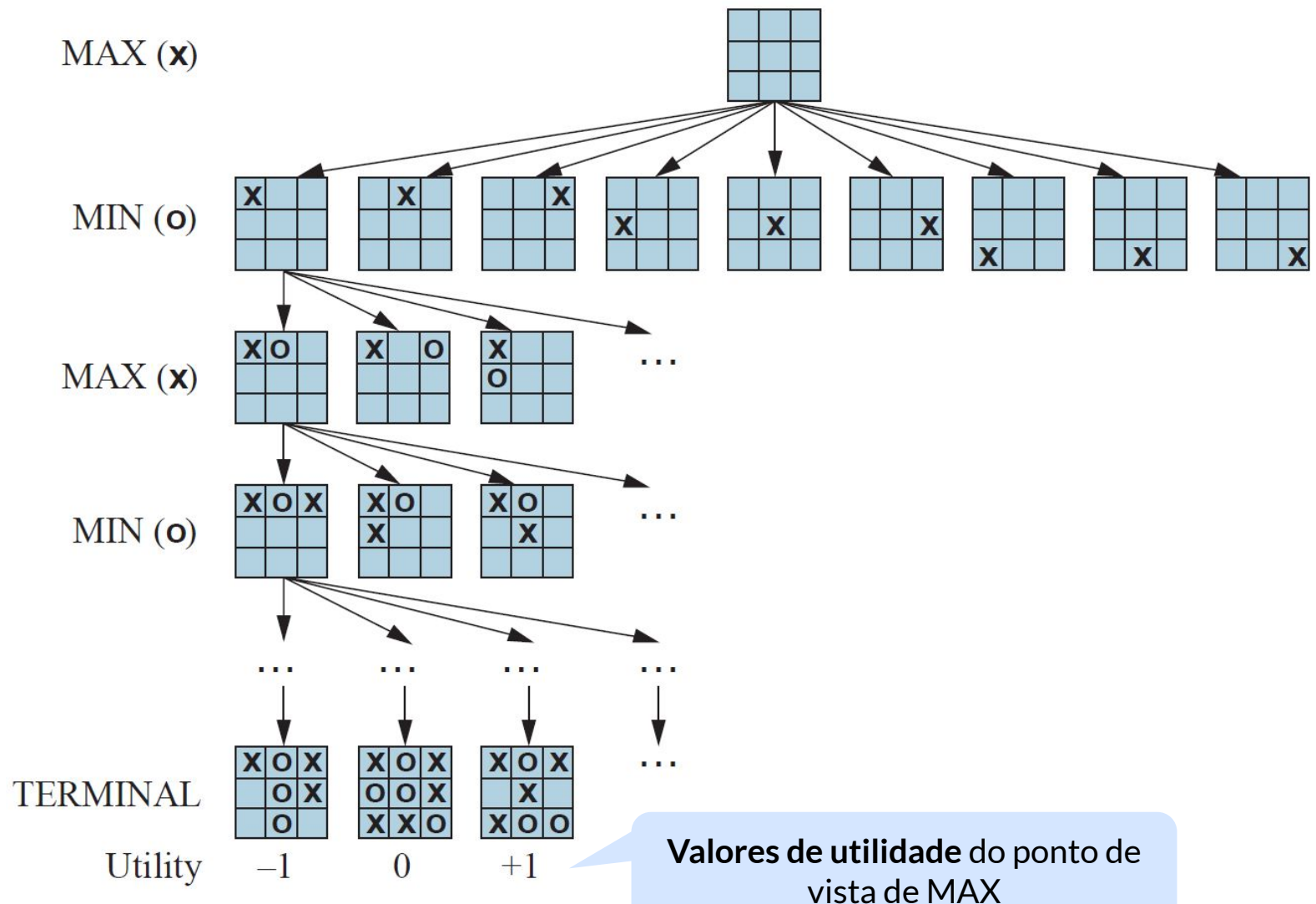
Considerando um jogo com dois jogadores: **MAX** e **MIN**

- ▷ MAX faz o primeiro movimento e depois ambos vão alternando as jogadas
- ▷ No fim do jogo, pontos são dados ao vencedor e o perdedor é penalizado

Formulação do Problema

1. **Estado inicial** em que o jogo é inicializado
2. **Jogador que deve realizar uma ação** no estado s
3. **Conjunto de ações válidas** no estado s
4. **Modelo de transição** que define o resultado de uma ação a no estado s
5. **Teste de estado terminal** que verifica se o jogo acabou
6. **Função de utilidade** que define um valor numérico para um jogador p quando o jogo termina no estado s
 - No xadrez, valores são 1 (vitória), 0 (derrota) e $\frac{1}{2}$ (empate)

Exemplo: Jogo da Velha



Decisões Ótimas em Jogos

- ▷ **Problema de busca normal** → solução ótima é sequência de ações que leva a um objetivo
- ▷ Em um jogo, as ações de MIN tem relação com esse objetivo (do ponto de vista de MAX)
 - MAX deve então encontrar uma **estratégia de contingência** que especifique o movimento de MAX no estado inicial e depois os movimentos de MAX nos estados resultantes de cada resposta possível de MIN, e assim por diante

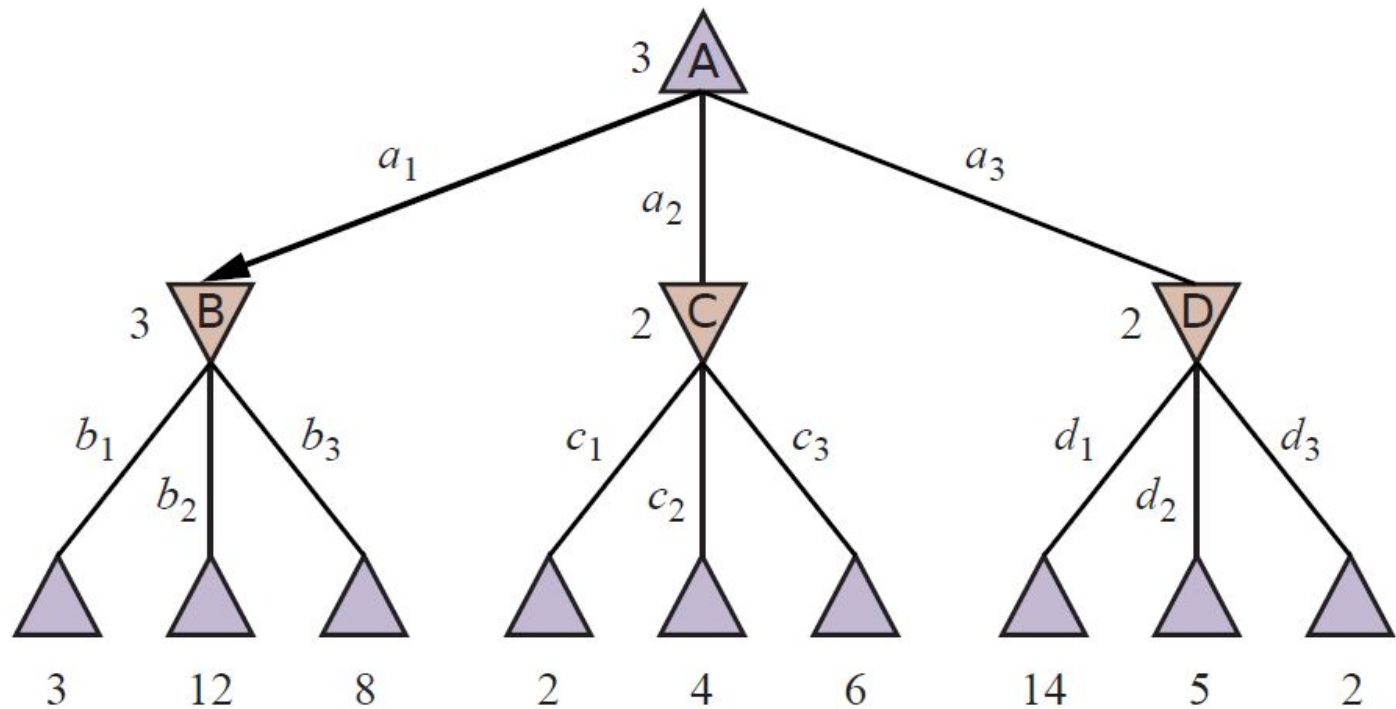
Decisões Ótimas em Jogos

- ▷ Dada uma árvore de jogo, a estratégia ótima pode ser determinada a partir do **valor minimax** de cada nó
- ▷ O valor minimax (para MAX) de é o resultado que ele pode garantir a partir dali, **assumindo que ele joga da melhor forma possível e o adversário também**
 - Ou seja, cada posição da árvore recebe um valor que mostra o “melhor resultado possível” para MAX, considerando que o oponente vai sempre tentar atrapalhar ao máximo

Decisões Ótimas em Jogos

MAX

MIN

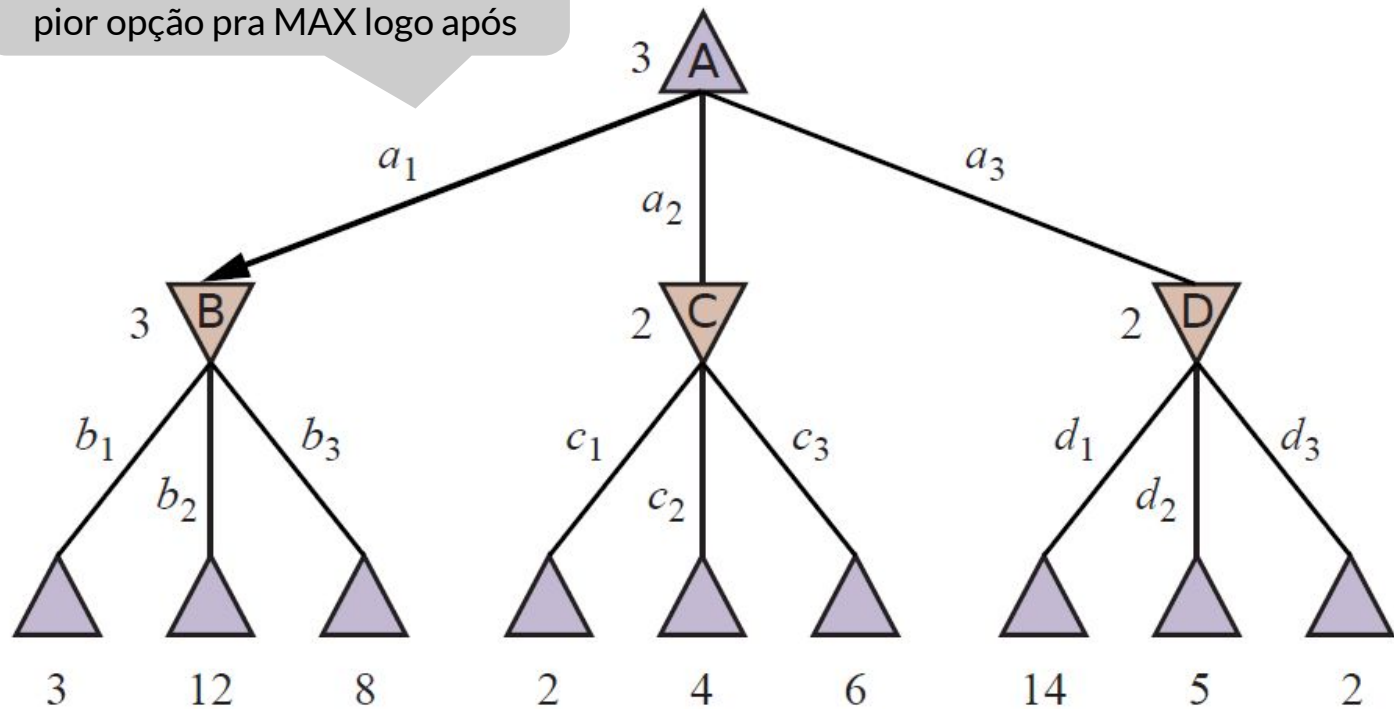


Decisões Ótimas em Jogos

MAX escolhe a melhor opção sabendo que MIN vai escolher a pior opção pra MAX logo após

MAX

MIN

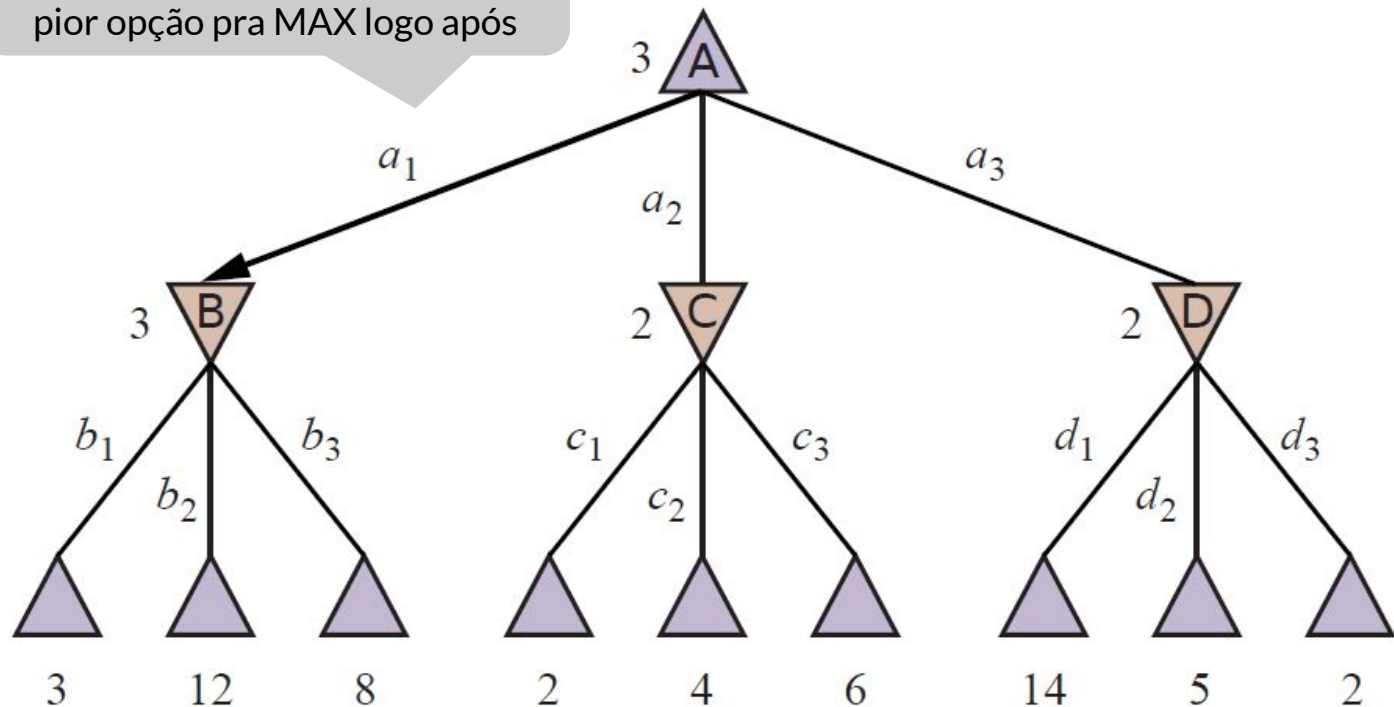


Decisões Ótimas em Jogos

MAX escolhe a melhor opção sabendo que MIN vai escolher a pior opção pra MAX logo após

MAX

MIN



MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

2.

Algoritmo MINIMAX

Como calcular?

Algoritmo MINIMAX

Intuição:

1. Gera a árvore inteira até os estados terminais
2. Aplica a função de utilidade nas folhas (terminais)
3. Vai subindo a árvore propagando os valores minimax de volta
4. Ao chegar na raiz, escolhe a ação que será realizada por MAX

Algoritmo MINIMAX

function MINIMAX-SEARCH(*game, state*) **returns** *an action*

$\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$

$\text{value, move} \leftarrow \text{MAX-VALUE}(\text{game, state})$

return *move*

function MAX-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game.IS-TERMINAL(state)* **then return** *game.UTILITY(state, player), null*

$v \leftarrow -\infty$

for each *a in game.ACTIONS(state)* **do**

$v2, a2 \leftarrow \text{MIN-VALUE}(\text{game, game.RESULT}(\text{state}, a))$

if $v2 > v$ **then**

$v, \text{move} \leftarrow v2, a$

return *v, move*

function MIN-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game.IS-TERMINAL(state)* **then return** *game.UTILITY(state, player), null*

$v \leftarrow +\infty$

for each *a in game.ACTIONS(state)* **do**

$v2, a2 \leftarrow \text{MAX-VALUE}(\text{game, game.RESULT}(\text{state}, a))$

if $v2 < v$ **then**

$v, \text{move} \leftarrow v2, a$

return *v, move*

Algoritmo MINIMAX

```
function MINIMAX-SEARCH(game, state) returns an action  
  player  $\leftarrow$  game.TO-MOVE(state)  
  value, move  $\leftarrow$  MAX-VALUE(game, state)  
  return move
```

```
function MAX-VALUE(game, state) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow -\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a))  
    if v2 > v then  
      v, move  $\leftarrow$  v2, a  
  return v, move
```

Decide qual é a melhor jogada a fazer no estado atual

1. Chama a função que calcula o melhor valor possível (MAX-VALUE)
2. Retorna a ação ideal

```
function MIN-VALUE(game, state) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow +\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a))  
    if v2 < v then  
      v, move  $\leftarrow$  v2, a  
  return v, move
```

Algoritmo MINIMAX

```
function MINIMAX-SEARCH(game, state) return  
  player  $\leftarrow$  game.TO-MOVE(state)  
  value, move  $\leftarrow$  MAX-VALUE(game, state)  
  return move
```

Representa o jogador MAX

1. Testa todas as jogadas possíveis
2. Escolhe a jogada com **maior valor** (melhor resultado)
3. Assume que MIN jogará da melhor forma contra ele

```
function MAX-VALUE(game, state) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow -\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a))  
    if v2 > v then  
      v, move  $\leftarrow$  v2, a  
  return v, move
```

```
function MIN-VALUE(game, state) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow +\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a))  
    if v2 < v then  
      v, move  $\leftarrow$  v2, a  
  return v, move
```

Algoritmo MINIMAX

```
function MINIMAX-SEARCH(game, state) returns an action  
  player  $\leftarrow$  game.TO-MOVE(state)  
  value, move  $\leftarrow$  MAX-VALUE(game, state)  
  return move
```

```
function MAX-VALUE(game, state) returns a (utility, move) pair
```

```
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player)  
  v  $\leftarrow -\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a))  
    if v2 > v then  
      v, move  $\leftarrow$  v2, a  
  return v, move
```

Representa o jogador MIN

1. Testa todas as jogadas possíveis
2. Escolhe a jogada com **menor valor** (pior resultado para MAX)
3. Assume que MAX também jogará da melhor forma

```
function MIN-VALUE(game, state) returns a (utility, move) pair
```

```
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow +\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a))  
    if v2 < v then  
      v, move  $\leftarrow$  v2, a  
  return v, move
```

Avaliação do algoritmo MINIMAX

- ▷ Equivale a uma **busca em profundidade** na árvore com profundidade m e com b ações para cada estado

Análise do algoritmo:

- ▷ **Completo:** sim (se árvore é finita)
- ▷ **Ótimo:** sim (oponente também é ótimo)
- ▷ **Complexidade:**
 - Tempo: $O(b^m)$
 - Espaço: $O(bm)$

Exemplo: Retirar palitos

- ▷ Há um monte com **5 palitos**
- ▷ Dois jogadores (MAX e MIN) jogam alternadamente
- ▷ Em cada turno, o jogador pode retirar 1, 2 ou 3 palitos
- ▷ O jogador que retirar o último palito perde

Será que MAX pode vencer?



 MAX joga

 MIN joga



 MAX joga

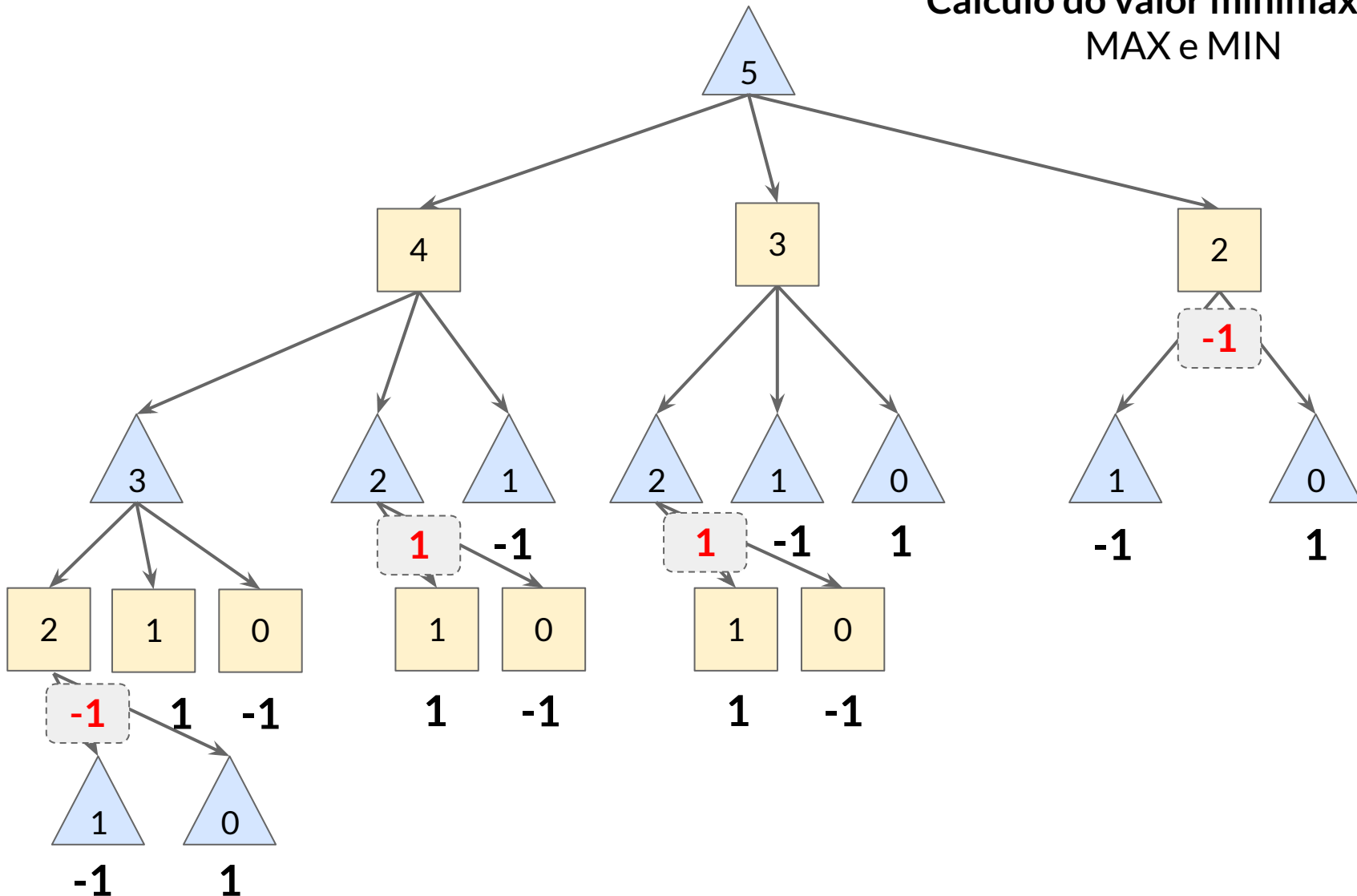
 MIN joga



Exemplo: Retirar palitos

△ MAX joga
□ MIN joga

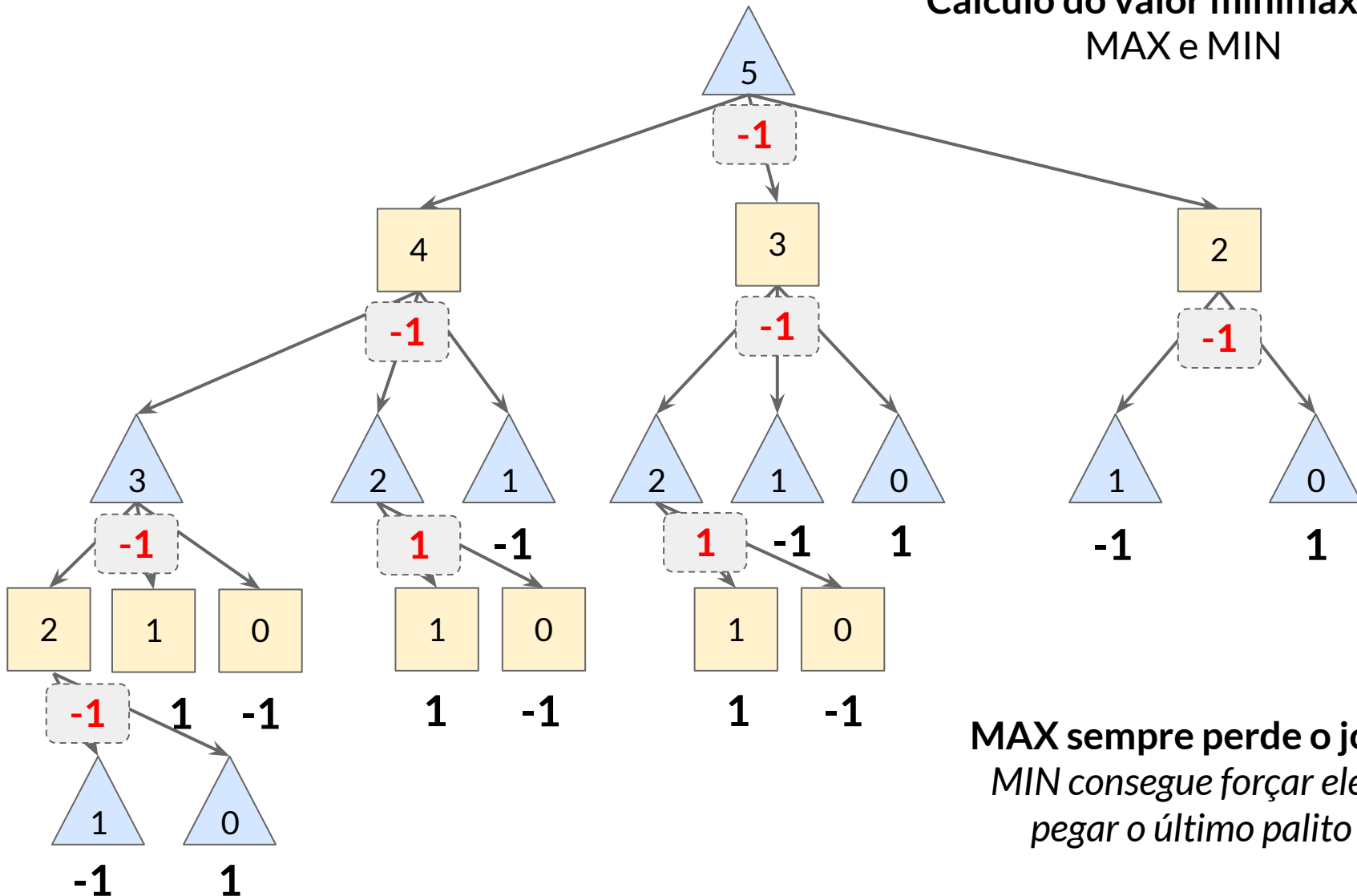
Cálculo do valor minimax para
MAX e MIN



Exemplo: Retirar palitos

△ MAX joga
□ MIN joga

Cálculo do valor minimax para
MAX e MIN



MAX sempre perde o jogo
MIN consegue forçar ele a
pegar o último palito

 MAX joga

 MIN joga

Cálculo do valor minimimax

MAX e MIN

MAX sempre perde o MIN consegue forçar e pegar o último palito

MAX sempre perde o jogo
*MIN consegue forçar ele a
 pegar o último palito*

MINIMAX: Problemas

- ▷ Tempo gasto para determinar a decisão ótima (ir até as folhas) é **impraticável**
- ▷ Complexidade exponencial $O(b^m)$

Ideias de melhoria:

MINIMAX: Problemas

- ▷ Tempo gasto para determinar a decisão ótima (ir até as folhas) é **impraticável**
- ▷ Complexidade exponencial $O(b^m)$

Ideias de melhoria:

- ▷ Limitar profundidade da busca e substituir função de utilidade por heurística
- ▷ Podar onde busca é irrelevante → **poda alfa-beta**

3.

Poda Alfa-Beta

Como melhorar a busca MINIMAX

Poda Alfa-Beta

Objetivo: desconsiderar partes (ramificações) da árvore que não terão influência possível sobre a decisão final

- ▷ **Ideia:** não vale a pena piorar, se já achou algo melhor

Considera dois parâmetros extra no MINIMAX:

- ▷ **α :** o melhor valor (mais alto) que o MAX já garantiu em algum ponto → “no mínimo”
- ▷ **β :** o melhor valor (mais baixo) que o MIN já garantiu em algum ponto → “no máximo”

Poda Alfa-Beta

1. A árvore é percorrida como no algoritmo MINIMAX
2. Em cada nó, verifica-se se ainda vale a pena analisar os próximos ramos
 - a. **Poda Alfa:** em um nó MIN, se $\alpha \geq \beta$, os demais ramos são ignorados
 - b. **Poda Beta:** em um nó MAX, se $\beta \leq \alpha$, os demais ramos são ignorados

Assim, o algoritmo evita cálculos desnecessários e fica mais rápido

Poda Alfa-Beta

1. A árvore é percorrida como no algoritmo MINIMAX
2. Em cada nó, verifica-se se ainda vale a pena analisar os próximos ramos
 - a. **Poda Alfa:** em um nó MIN, se $\alpha \geq \beta$, os demais ramos são ignorados
 - b. **Poda Beta:** em um nó MAX, se $\beta \leq \alpha$, os demais ramos são ignorados

Assim, o algoritmo evita cálculos desnecessários e fica mais rápido

Como assim?

Poda Alfa (corte no MIN)

Cenário: você está em um nó MIN avaliando o valor β

Pergunta: “Existe algum ancestral MAX que já tenha uma opção melhor que o meu β atual?”

Lógica: Se α (ancestral) $\geq \beta$ (atual), o MAX nunca escolherá este caminho

Ação: Pare de olhar os outros filhos deste nó. **Poda!**

“Se eu (MIN) encontrar algo pior do que o meu pai (MAX) já tem garantido, ele nem vai olhar o resto deste caminho.”

Poda Beta (corte no MAX)

Cenário: você está em um nó MAX avaliando o valor α

Pergunta: “Existe algum ancestral MIN que já tenha uma opção ‘menos pior’ que o meu α atual?”

Lógica: Se β (ancestral) $\leq \alpha$ (atual), o MIN nunca escolherá este caminho

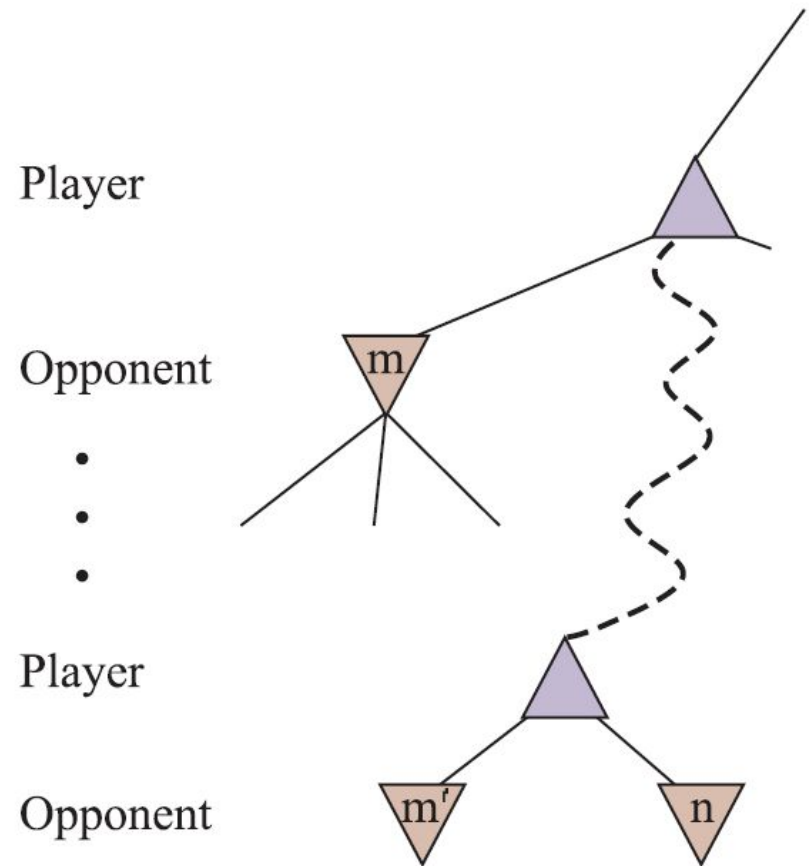
Ação: Pare de olhar os outros filhos deste nó. **Poda!**

“Se eu (MAX) encontrar algo melhor do que o meu pai (MIN) já aceitou, ele vai descartar este caminho imediatamente.”

Poda Alfa-Beta

Caso geral da poda

Se m ou m' é melhor do que n para o jogador (MAX), nunca se chegará no estado n no jogo



Poda Alfa-Beta

function ALPHA-BETA-SEARCH(*game, state*) **returns** an action

$\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$

$\text{value, move} \leftarrow \text{MAX-VALUE}(\text{game, state, } -\infty, +\infty)$

return *move*

function MAX-VALUE(*game, state, α, β*) **returns** a (*utility, move*) pair

if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state, player*), *null*

$v \leftarrow -\infty$

for each *a* **in** *game.ACTIONS*(*state*) **do**

$v2, a2 \leftarrow \text{MIN-VALUE}(\text{game, game.RESULT}(\text{state, a}), \alpha, \beta)$

if $v2 > v$ **then**

$v, \text{move} \leftarrow v2, a$

$\alpha \leftarrow \text{MAX}(\alpha, v)$

if $v \geq \beta$ **then return** *v, move*

return *v, move*

function MIN-VALUE(*game, state, α, β*) **returns** a (*utility, move*) pair

if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state, player*), *null*

$v \leftarrow +\infty$

for each *a* **in** *game.ACTIONS*(*state*) **do**

$v2, a2 \leftarrow \text{MAX-VALUE}(\text{game, game.RESULT}(\text{state, a}), \alpha, \beta)$

if $v2 < v$ **then**

$v, \text{move} \leftarrow v2, a$

$\beta \leftarrow \text{MIN}(\beta, v)$

if $v \leq \alpha$ **then return** *v, move*

return *v, move*

Poda Alfa-Beta

```
function ALPHA-BETA-SEARCH(game, state) returns an action  
  player  $\leftarrow$  game.TO-MOVE(state)  
  value, move  $\leftarrow$  MAX-VALUE(game, state,  $-\infty$ ,  $+\infty$ )  
  return move
```

Mantém os limites nas
variáveis α e β

```
function MAX-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow -\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )  
    if v2 > v then  
      v, move  $\leftarrow$  v2, a  
       $\alpha \leftarrow$  MAX( $\alpha$ , v)  
    if v  $\geq \beta$  then return v, move  
  return v, move
```

```
function MIN-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow +\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )  
    if v2 < v then  
      v, move  $\leftarrow$  v2, a  
       $\beta \leftarrow$  MIN( $\beta$ , v)  
    if v  $\leq \alpha$  then return v, move  
  return v, move
```

Poda Alfa-Beta

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action
 $\text{player} \leftarrow \text{game.T0-MOVE}(\text{state})$
 $\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state}, -\infty, +\infty)$
 return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*
 $v \leftarrow -\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 > v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 if $v \geq \beta$ **then return** *v*, *move*
 return *v*, *move*



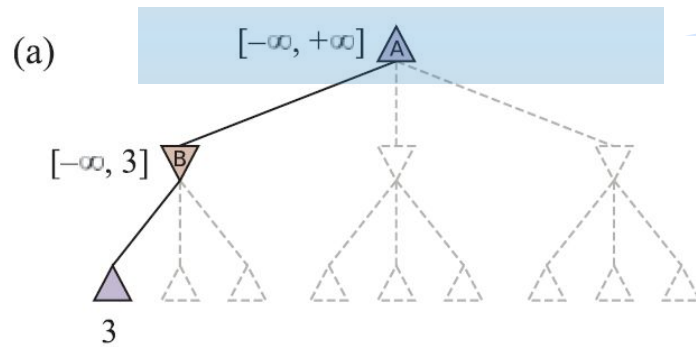
Poda Beta

function MIN-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*
 $v \leftarrow +\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MAX-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 < v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\beta \leftarrow \text{MIN}(\beta, v)$
 if $v \leq \alpha$ **then return** *v*, *move*
 return *v*, *move*



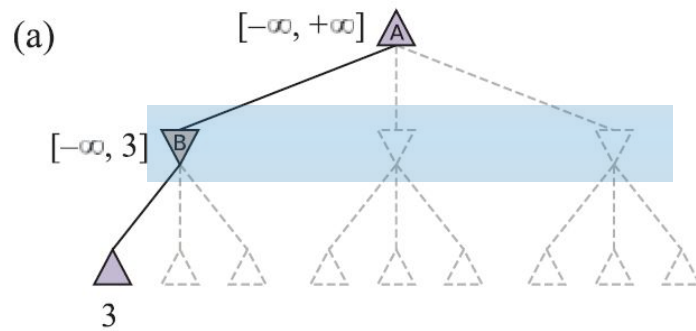
Poda Alfa

Poda Alfa-Beta



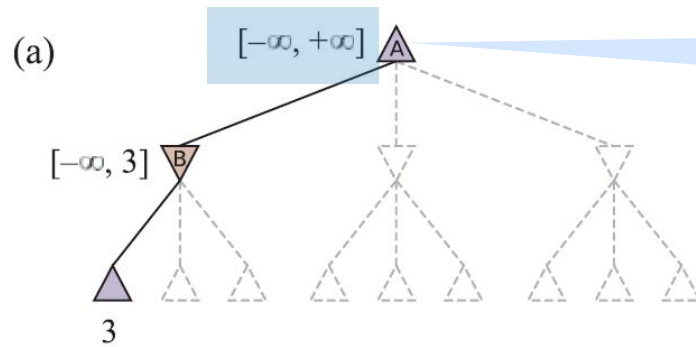
MAX: escolher o caminho com o maior valor possível (α é a sua melhor garantia de ganho)

Poda Alfa-Beta



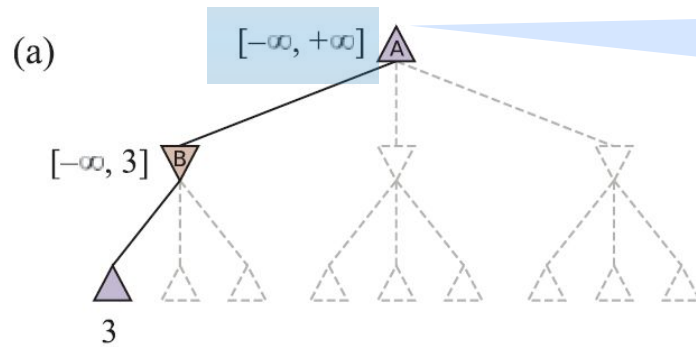
MIN: escolher o caminho com o menor valor possível (β é o máximo que ele vai permitir que o MAX ganhe)

Poda Alfa-Beta



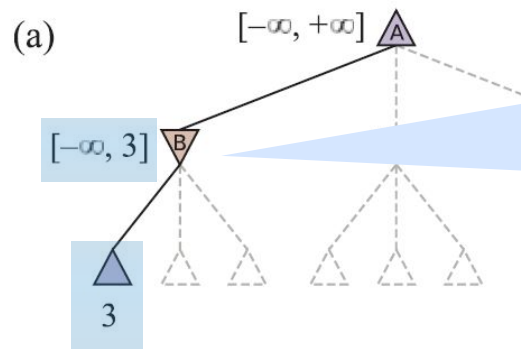
Intervalos $[\alpha, \beta]$: limites conhecidos no momento para aquele nó

Poda Alfa-Beta



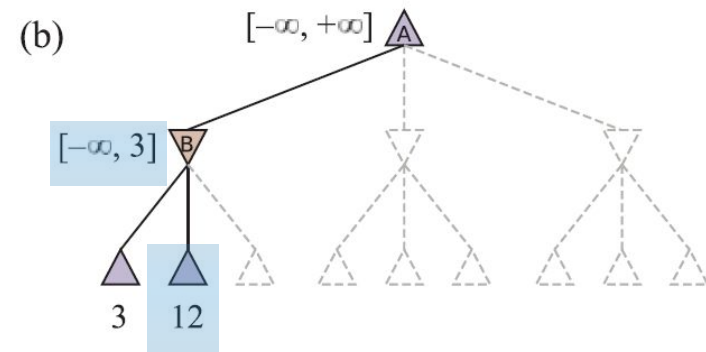
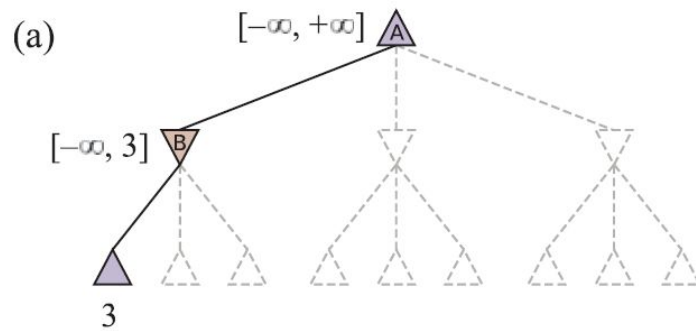
O nó A (MAX) começa **sem limites definidos** $[-\infty, +\infty]$ e desce para avaliar sua primeira opção, o nó B (MIN)

Poda Alfa-Beta



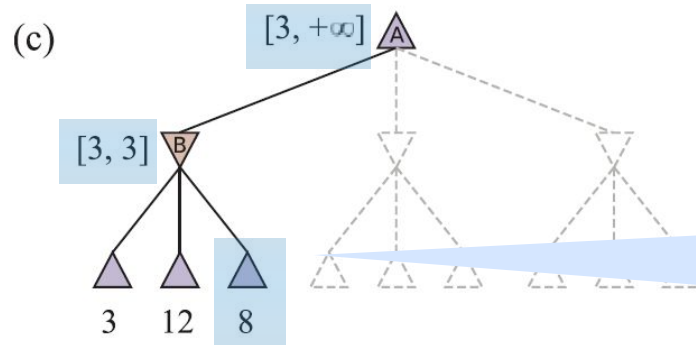
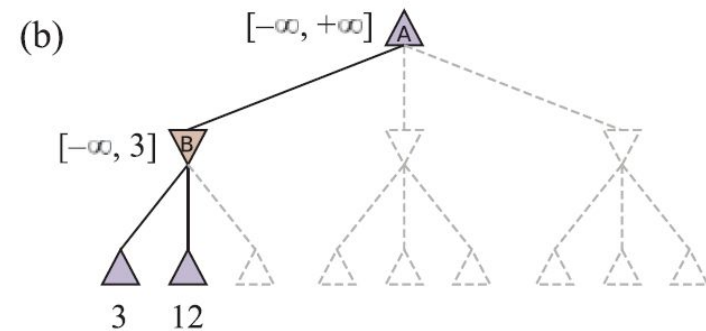
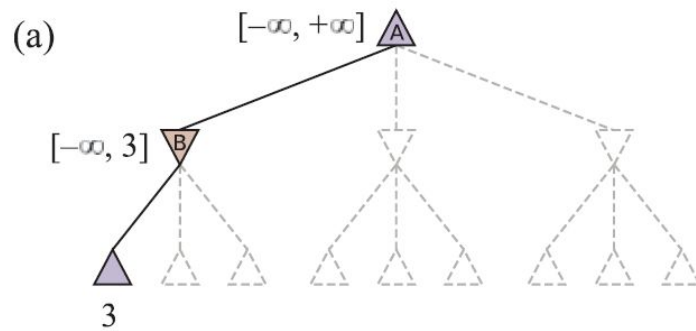
B avalia seu primeiro filho, que tem valor 3. Como ele é um nó MIN, B atualiza seu limite superior (β) para 3, pois ele já encontrou um caminho que limita o ganho de A a no máximo 3. O intervalo de B fica $[-\infty, 3]$

Poda Alfa-Beta



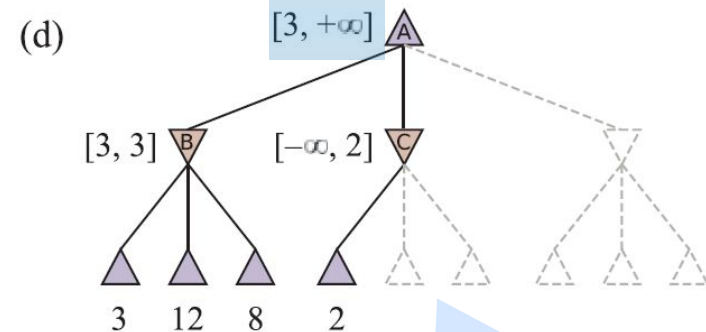
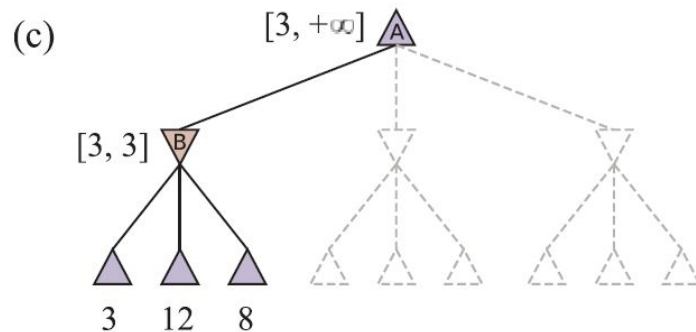
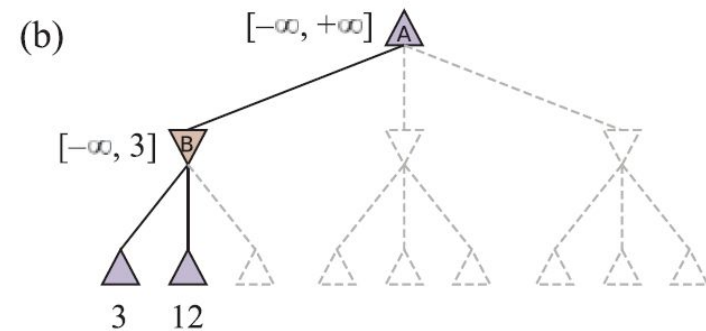
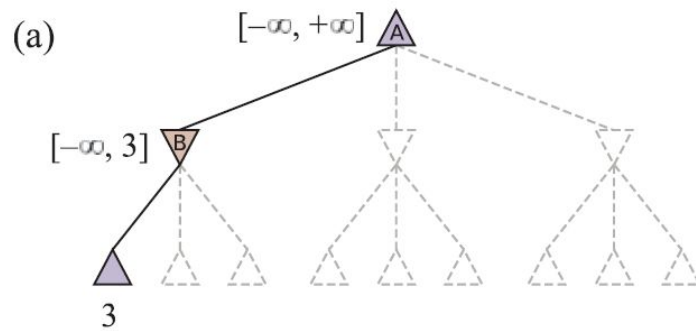
B avalia o próximo filho. Como 12 é maior que 3, B ignora, pois ele prefere números menores.

Poda Alfa-Beta



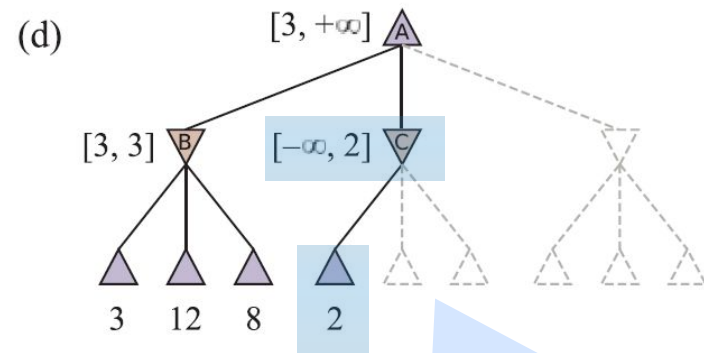
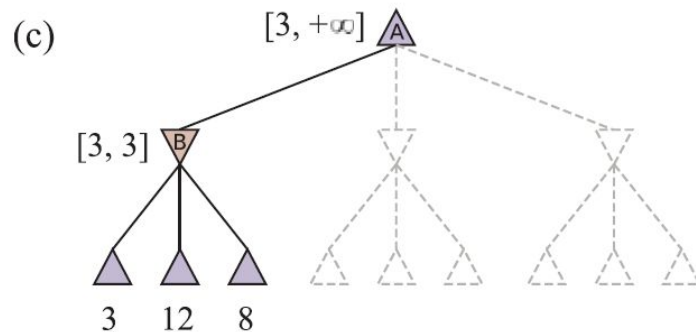
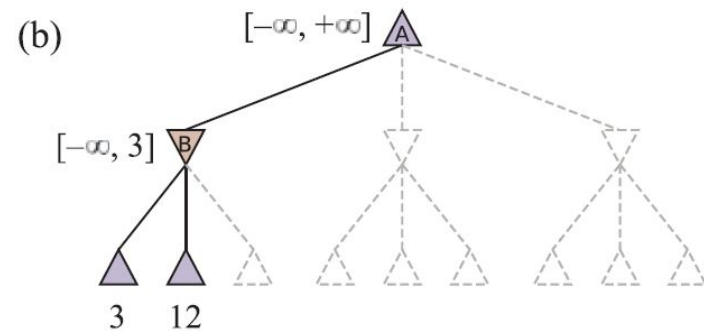
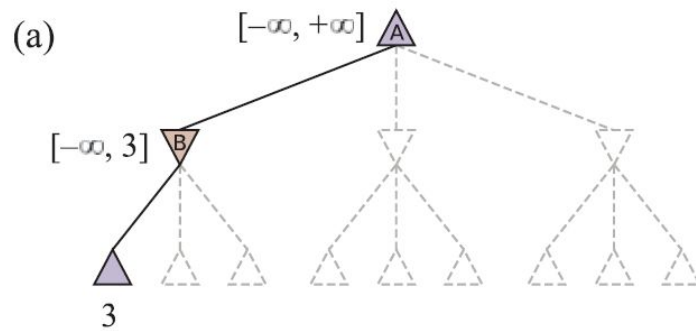
B avalia o último filho (8). Novamente, ignora. Tendo avaliado todos os filhos, B infere que o valor da raiz é pelo menos 3 e passa essa informação para A.

Poda Alfa-Beta



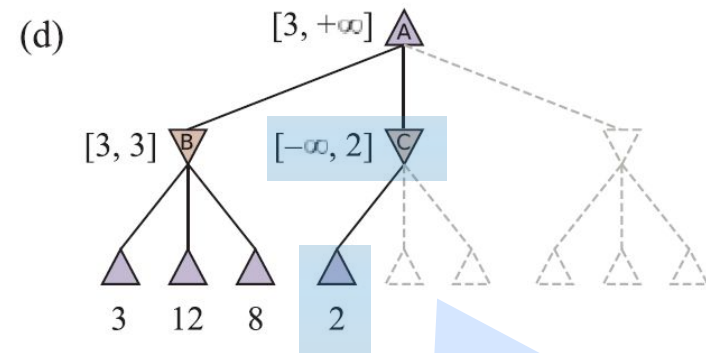
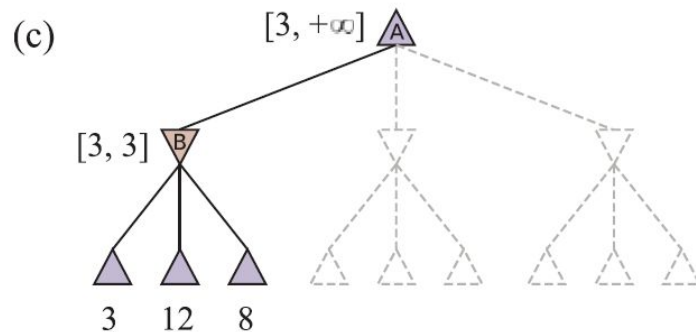
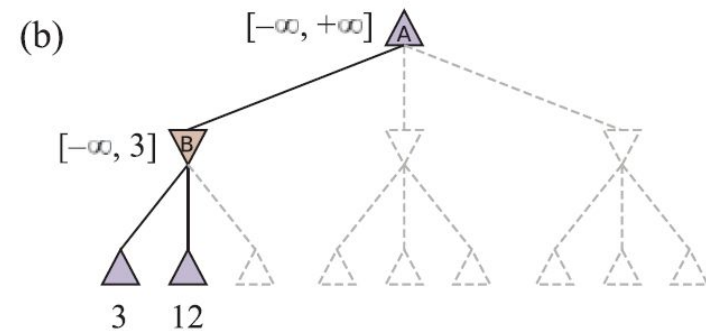
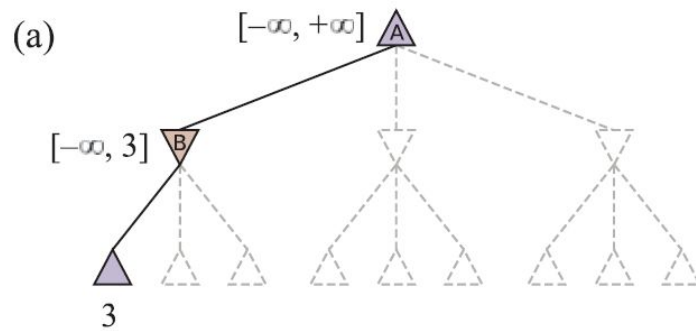
A busca vai para o nó C (MIN), passando a informação de que o MAX já tem 3 garantido

Poda Alfa-Beta



C avalia seu primeiro filho, que tem o valor 2. Como C é um nó MIN, ele imediatamente atualiza seu limite superior (β) para 2.

Poda Alfa-Beta



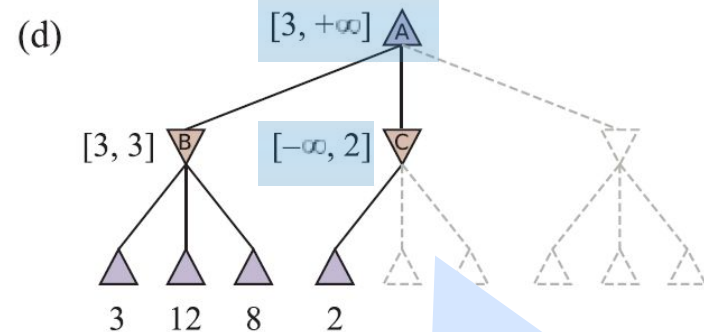
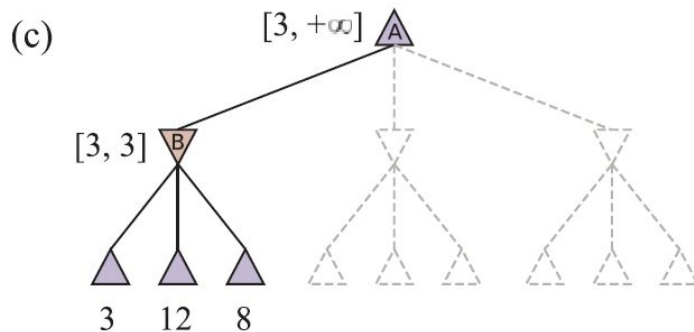
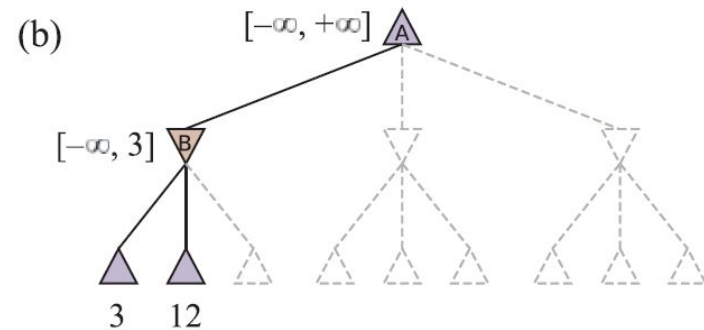
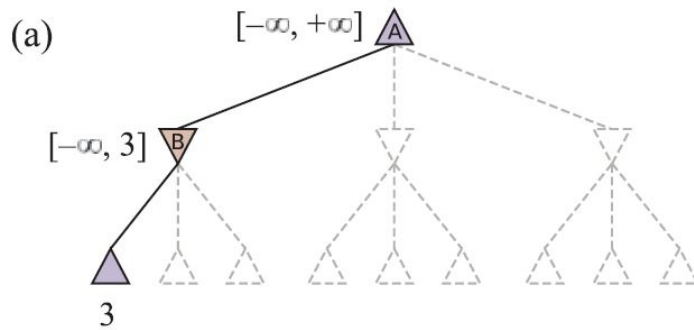
Ou seja, se o jogo vier por aqui, **C** vai garantir que o **MAX** ganhe no máximo 2 (ou menos, se os próximos filhos forem menores ainda)

Poda Alfa-Beta



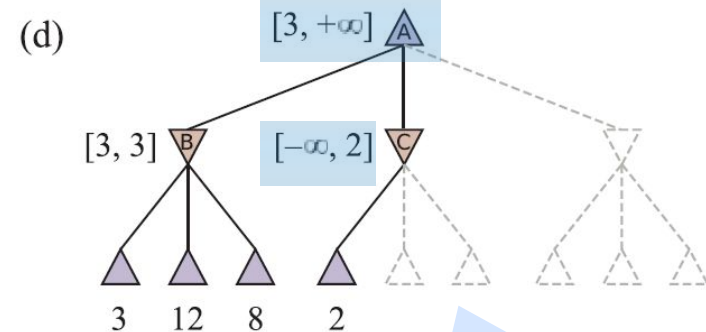
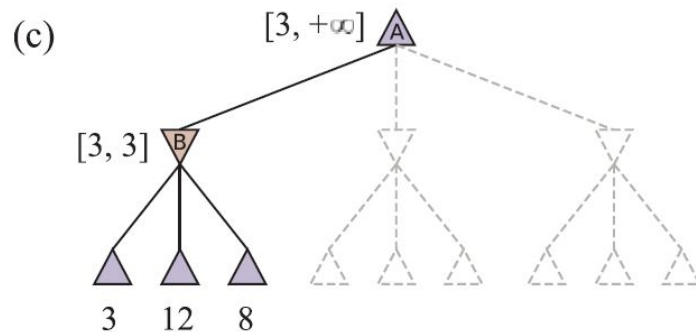
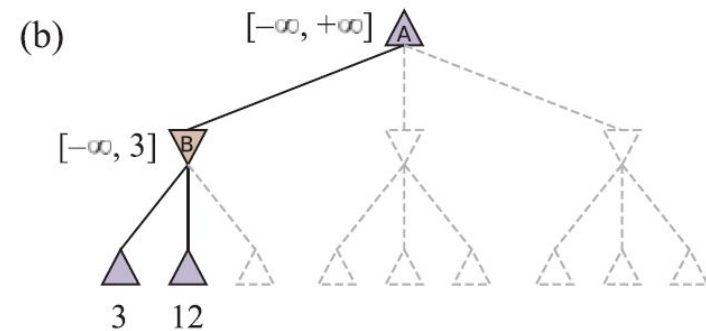
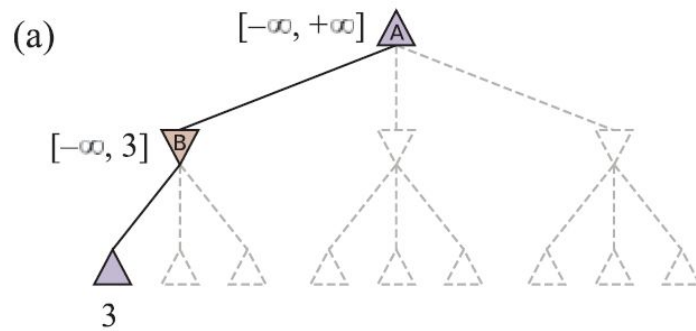
o
nos

Poda Alfa-Beta



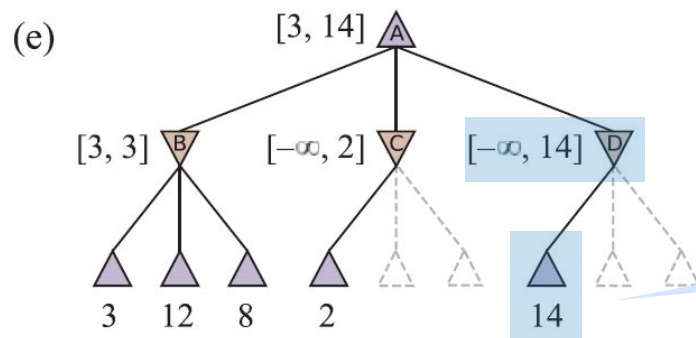
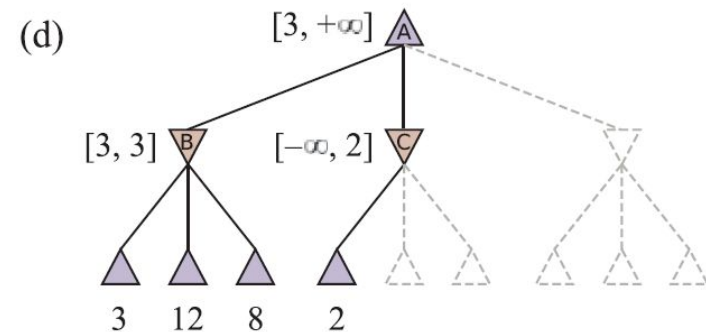
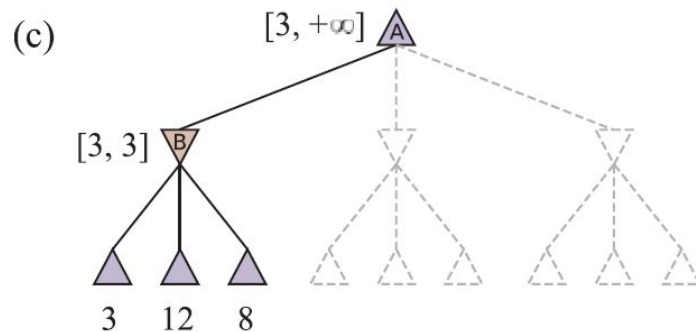
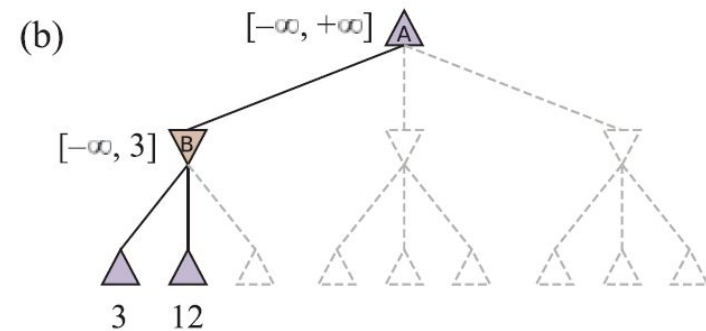
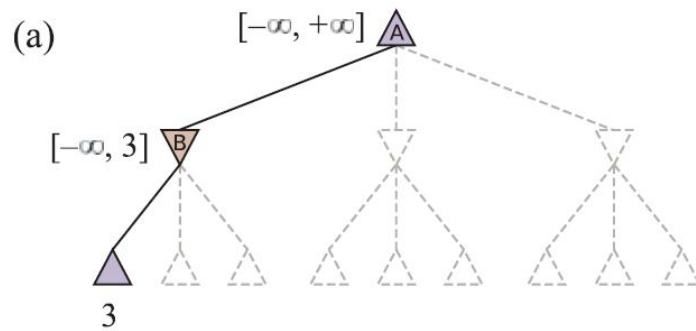
O nó A (MAX) é inteligente. Ele pensa: “Eu já tenho um caminho (nó B) que me garante 3. O caminho C acabou de mostrar que vai me dar no máximo 2, então eu nunca vou escolher o caminho C.”

Poda Alfa-Beta



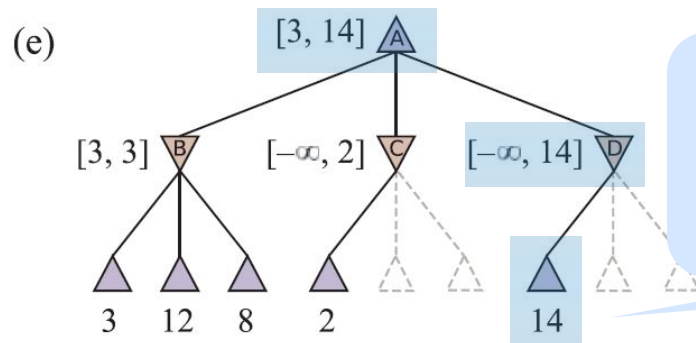
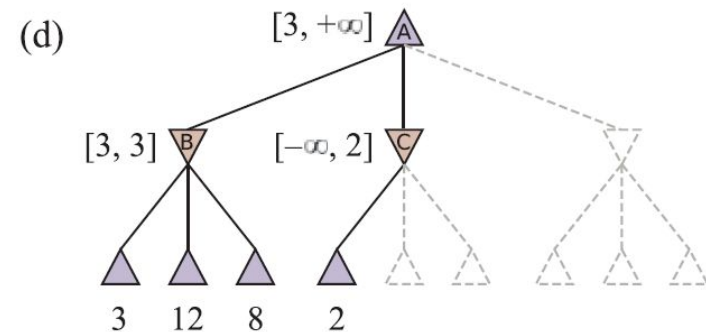
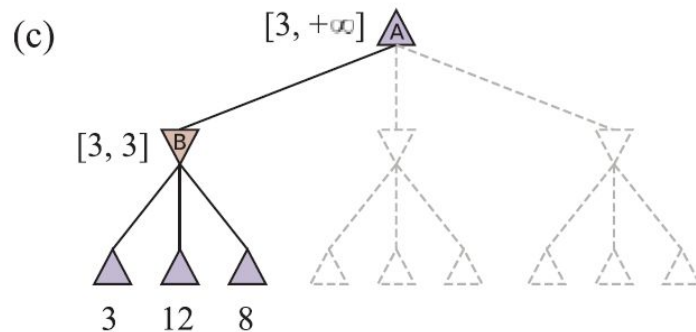
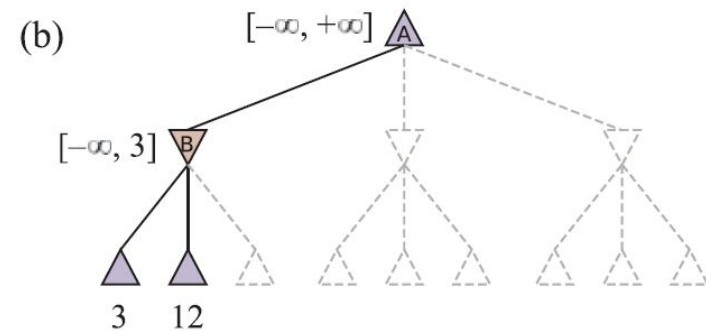
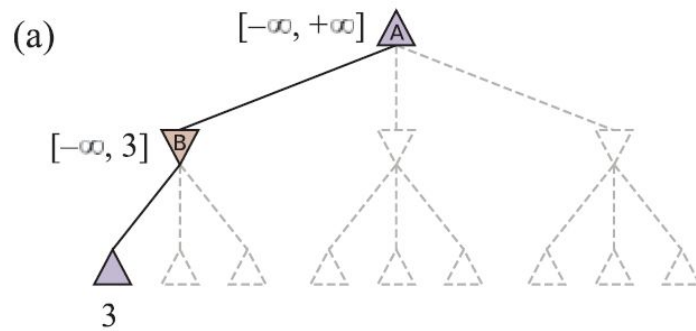
PODA ALFA! Como a condição de poda é atendida (β do nó atual $\leq \alpha$ do ancestral), o algoritmo simplesmente corta a avaliação dos outros dois filhos do nó C

Poda Alfa-Beta



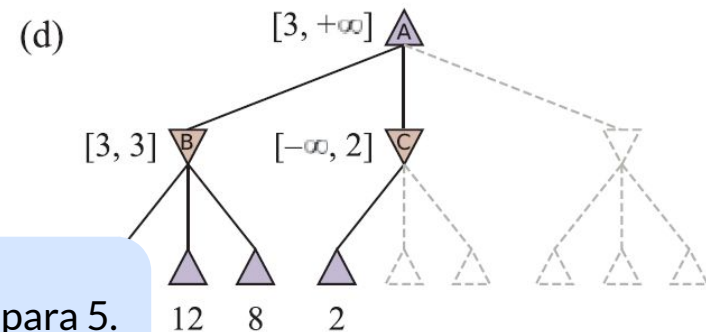
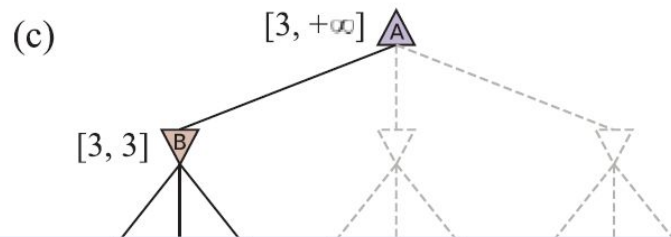
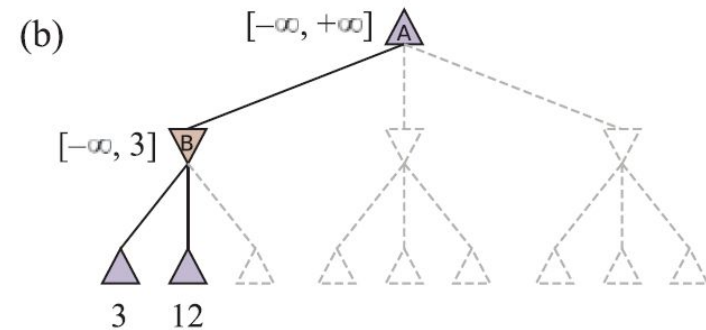
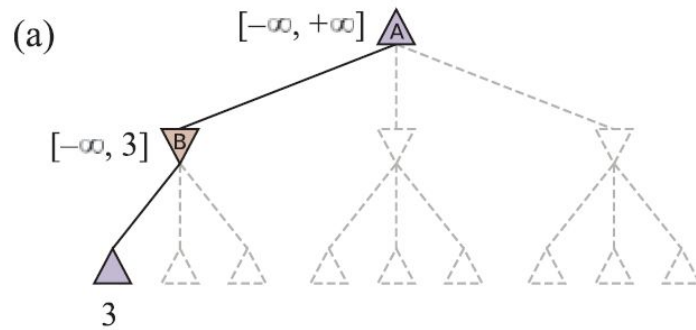
A busca desce para o nó D (MIN). O primeiro filho tem valor 14. D atualiza seu β para 14.

Poda Alfa-Beta

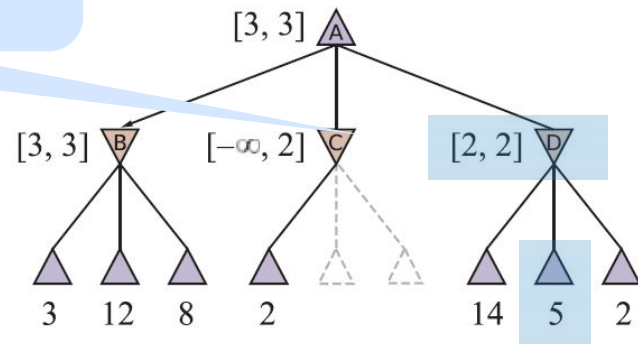
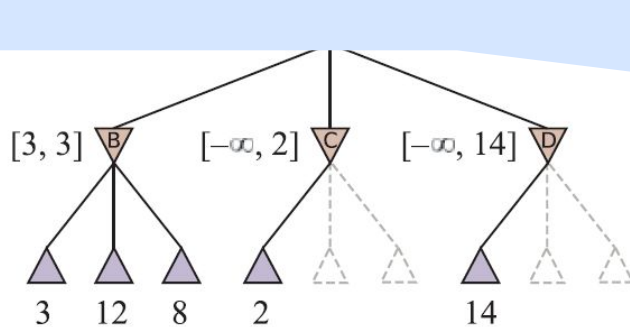


Como 14 é maior que o α de A (que é 3), **não podemos podar**. Ainda existe a chance de D encontrar um valor que seja menor que 14, mas ainda assim maior que 3. Portanto, a busca continua.

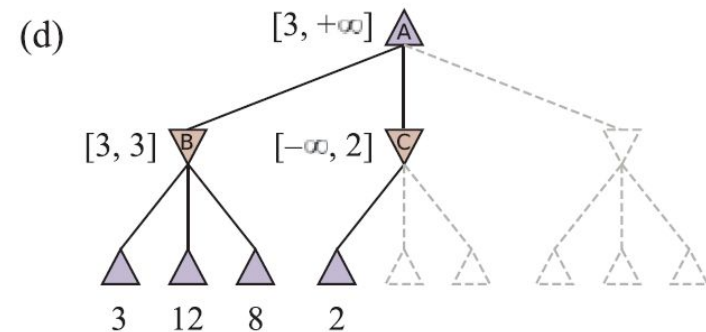
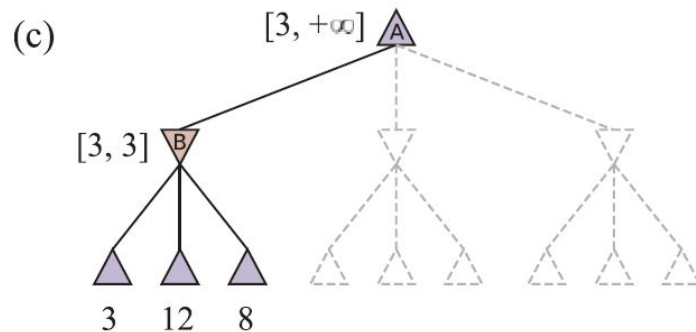
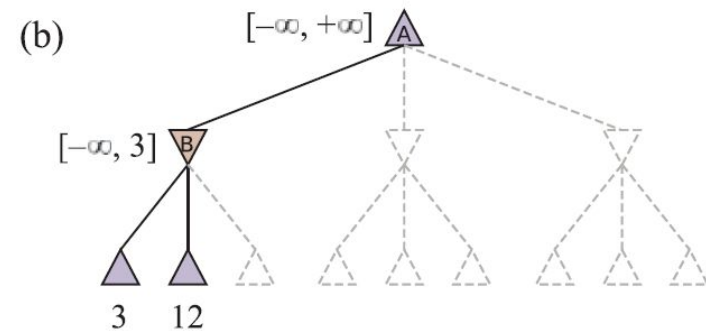
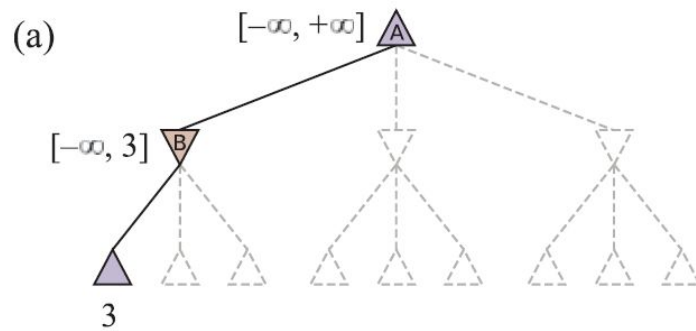
Poda Alfa-Beta



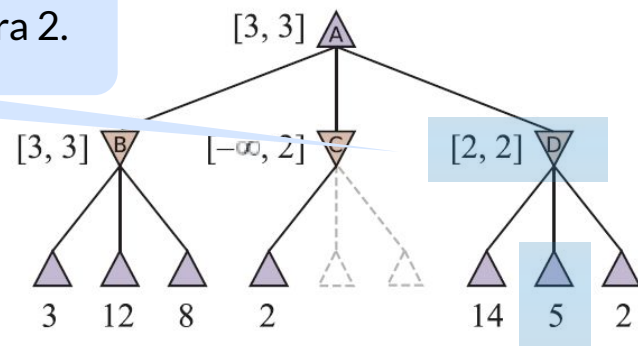
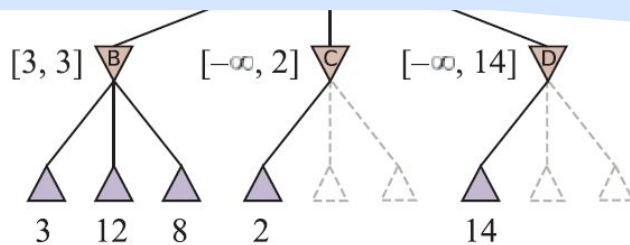
D avalia o próximo filho (5). Ele atualiza seu β para 5.
(Ainda não podemos podar, pois $5 > 3$).



Poda Alfa-Beta



D avalia o último filho (2). Ele atualiza seu β para 2.

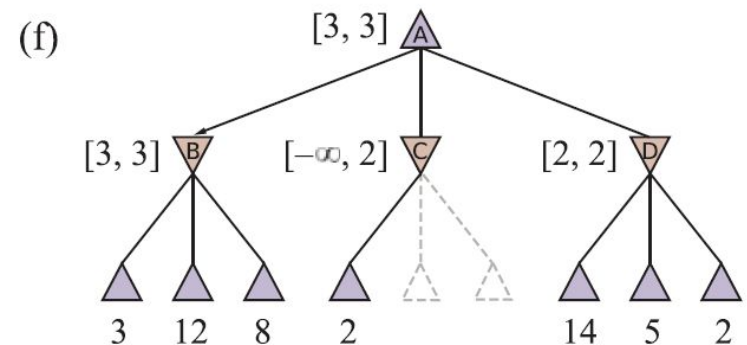


Poda Alfa-Beta

Resultado Final: O nó A (MAX) olha para as opções que os nós MIN o forçaram a aceitar

- ▷ Ir para B resulta em 3
- ▷ Ir para C resulta em 2 (ou pior, mas não importa, pois já foi podado)
- ▷ Ir para D resulta em 2

Escolha: Ir para B

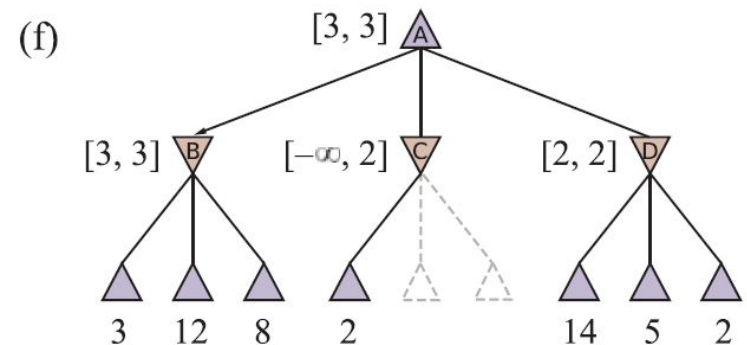


Poda Alfa-Beta

Resultado Final: O nó A (MAX) olha para as opções que os nós MIN o forçaram a aceitar

- ▷ Ir para B resulta em 3
- ▷ Ir para C resulta em 2 (ou pior, mas não importa, pois já foi podado)
- ▷ Ir para D resulta em 2

Escolha: Ir para B



Poda Alfa-Beta

- ▷ A efetividade da poda depende da **ordem em que os sucessores são analisados**
- ▷ Com a melhor ordem possível, a complexidade de tempo é $O(b^{m/2})$
 - Considerando fator de ramificação b e profundidade m
 - Melhor do que $O(b^m)$ do MINIMAX tradicional

Referências

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. 4th ed. Pearson, 2021. (*Capítulo 5*)